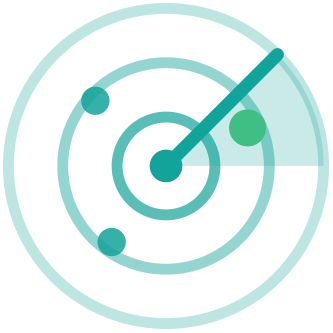


WhatsOnMyNet

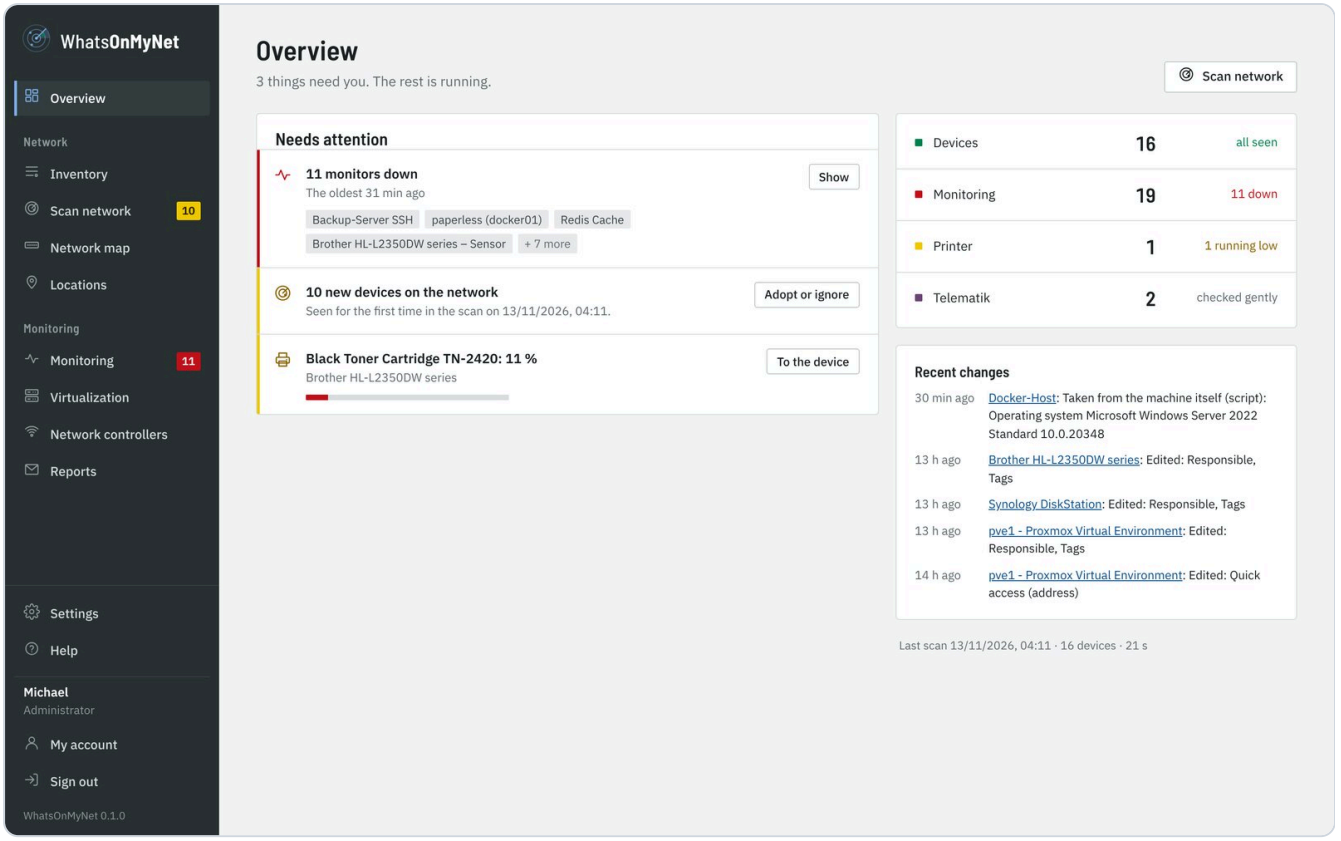
Manual - Network inventory and monitoring

2026-11-13

Contents

1. Installation
2. First steps
3. Inventory
4. Nightly light scan
5. Monitoring
6. Printers, switches and NAS
7. Sensors
8. Network drives (SMB)
9. Proxmox VE and Backup Server
10. More hypervisors: VMware, XCP-ng, Hyper-V, Nutanix, oVirt, KVM
11. Network controllers: UniFi, Omada, MikroTik
12. Windows and Linux machines by script (without agent)
13. Reports
14. Users and rights
15. Sign-in with Active Directory
16. Scan agent for branch offices
17. Backup and updates
18. Troubleshooting

WhatsOnMyNet finds what runs on your network and keeps only what you adopt in the inventory. It also monitors devices, services and websites, reads printers, switches, NAS and hypervisors (Proxmox, VMware, Hyper-V …), and reports by e-mail, messenger, push to the phone or webhook when something is wrong.



Overview

Installation

WhatsOnMyNet runs as a Docker container with host networking - only this way the scan sees the MAC addresses of the devices.

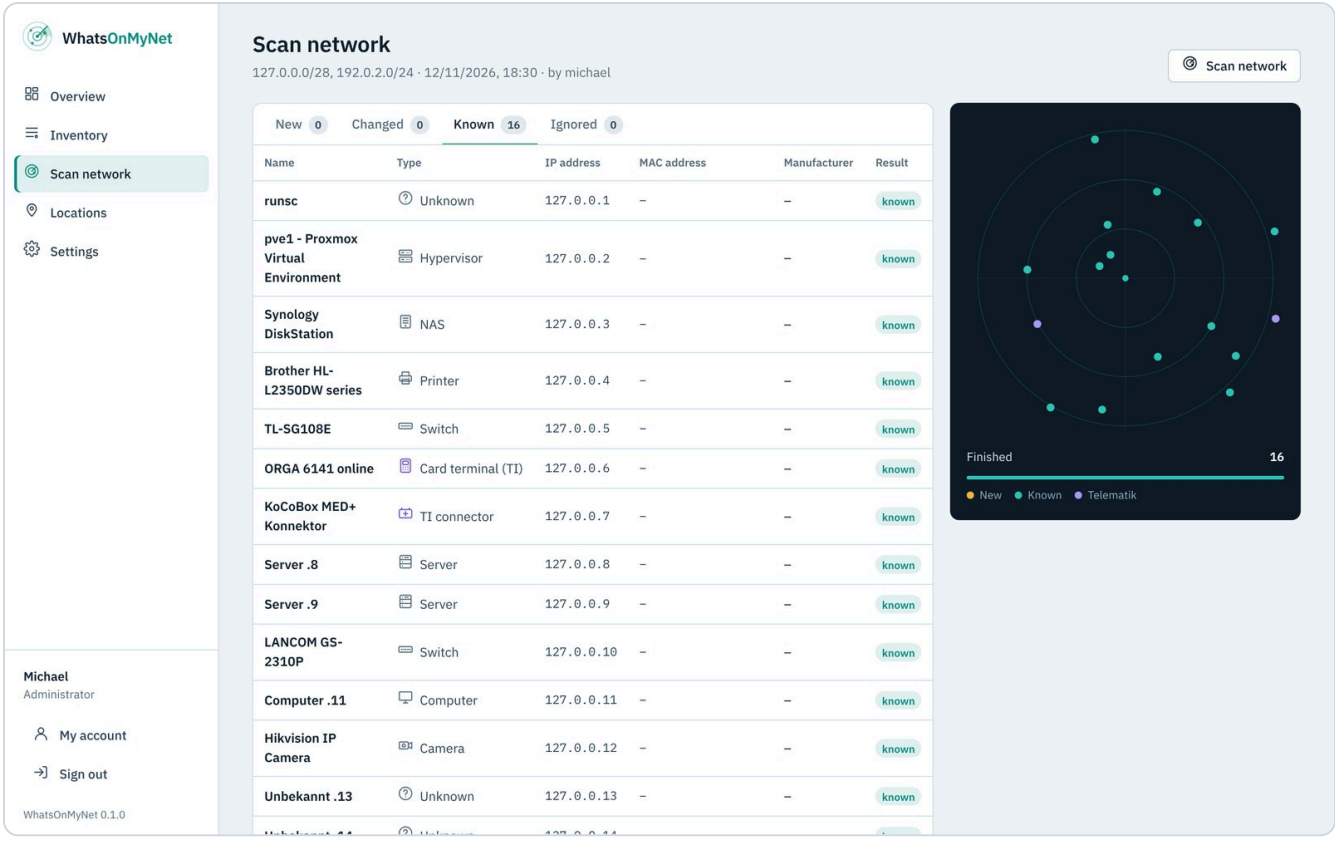
1. Put `docker-compose.yml` and `env.example` from the release into a folder, rename `env.example` to `.env`.
2. Adjust ports (`WOMN_HTTPS_PORT`, default 8743), time zone (`TZ`) and owner of the data (`PUID` / `PGID`) in `.env` if needed.
3. `docker compose up -d` - or create the stack in Dockhand.
4. Get the **setup code** from the log (`docker compose logs whatsonmynet`) and open `https://<server>:8743/setup`. There you create the first administrator.

The interface always runs over HTTPS. On first start WhatsOnMyNet creates a self-signed certificate; upload your own (PEM or PFX) under *Settings → HTTPS certificate*.

Ping: the container runs without root rights. Ping works if the host allows ICMP for normal users (`net.ipv4.ping_group_range`, default on Debian/Ubuntu/Proxmox; set `0 65535` in LXC containers). Without ping the scan still finds devices via ARP and TCP.

First steps

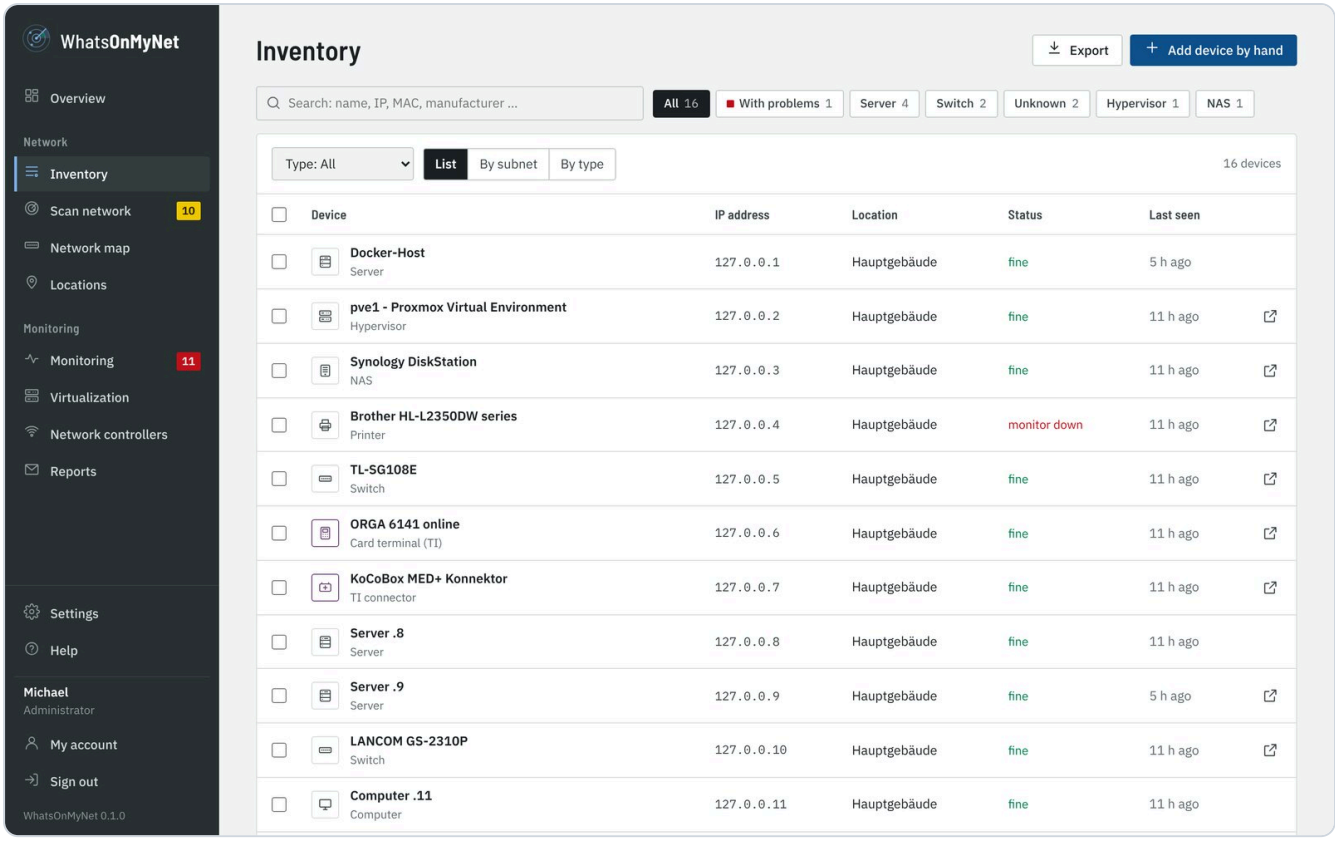
1. **Add a location** (*Locations*): name and subnets, e.g. "Main building" with `192.168.10.0/24`.
2. **Scan the network** (*Scan network*): tick the location, "Start scan". The radar shows the devices found; the scan is gentle (ARP, about 25 typical ports, SNMP read-only).
3. **Adopt**: in the result under "New" check type and name, tick devices, "Adopt into inventory". What you do not want in the inventory, "Ignore". Ignored by mistake? Tick it in the "Ignored" tab and choose "Stop ignoring" (it is listed under "New" again) or "Adopt into inventory" right away. The full list is under *Settings → Ignored devices*.



Scan with radar

The MAC address is the identity of a device: if it gets a new IP, the next scan takes it over automatically and records it in the history.

Inventory



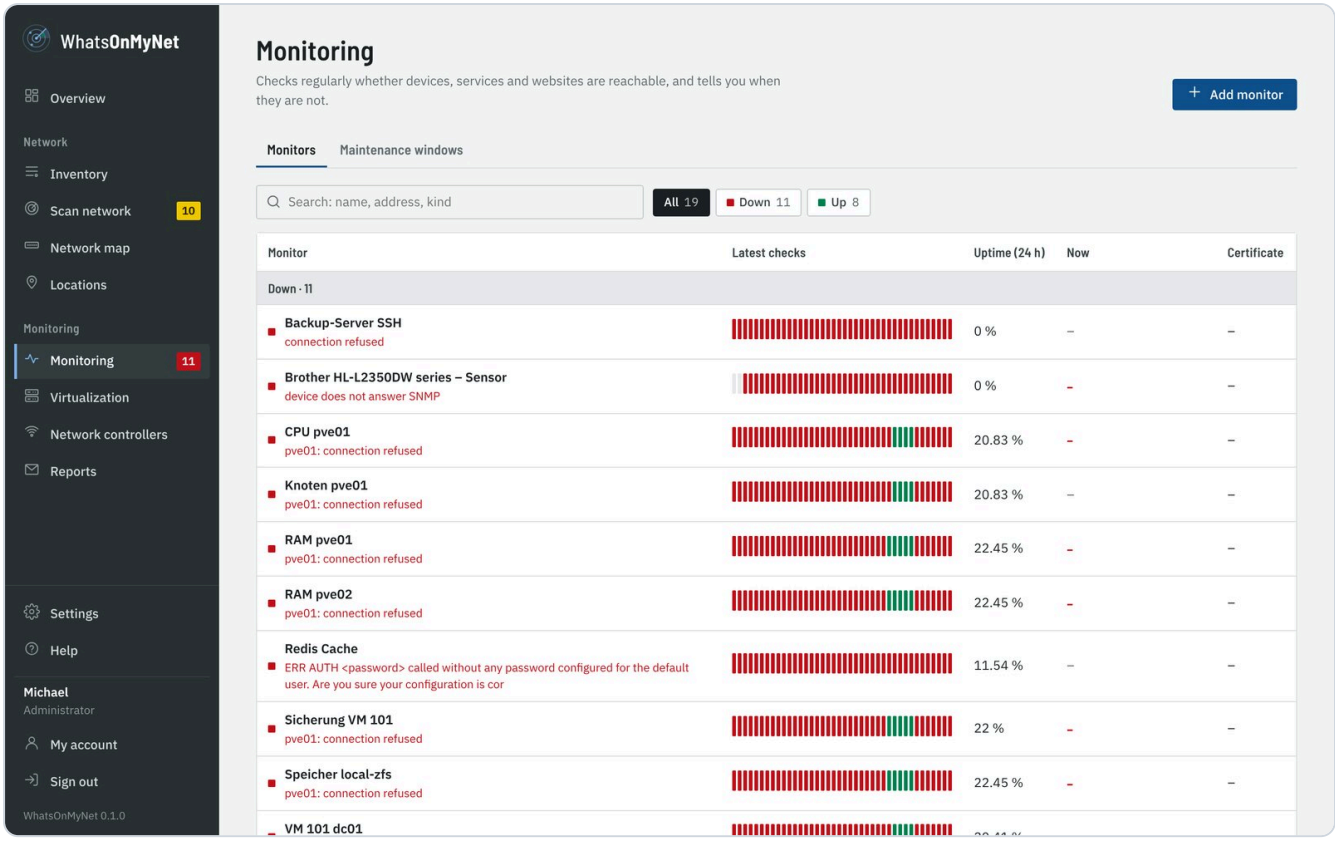
Inventory

- **Search and filter** by name, IP, MAC, manufacturer, type, location or tag; export as CSV or JSON.
- **Device page**: all details, purchase and warranty dates, notes, quick access to the web interface, what the scan detected, history of all changes.
- **Suggestions while typing**: manufacturer, model, operating system, owner, supplier and tags suggest what is already in the inventory. A known model fills in manufacturer and type.
- **Multiple selection**: tick devices and edit them together (type, location, manufacturer, model, dates, tags) or delete them.
- **Deep check**: checks one device with a longer port list. Telematik devices (connector, card terminal) are always checked gently.

Nightly light scan

Every night (default 02:30) WhatsOnMyNet looks for devices in all subnets of your locations - no ports, no SNMP. Known devices get their new IP; a device showing up for the very first time is reported once. Settings, time and "Run now" are under *Settings → Scan*.

Monitoring

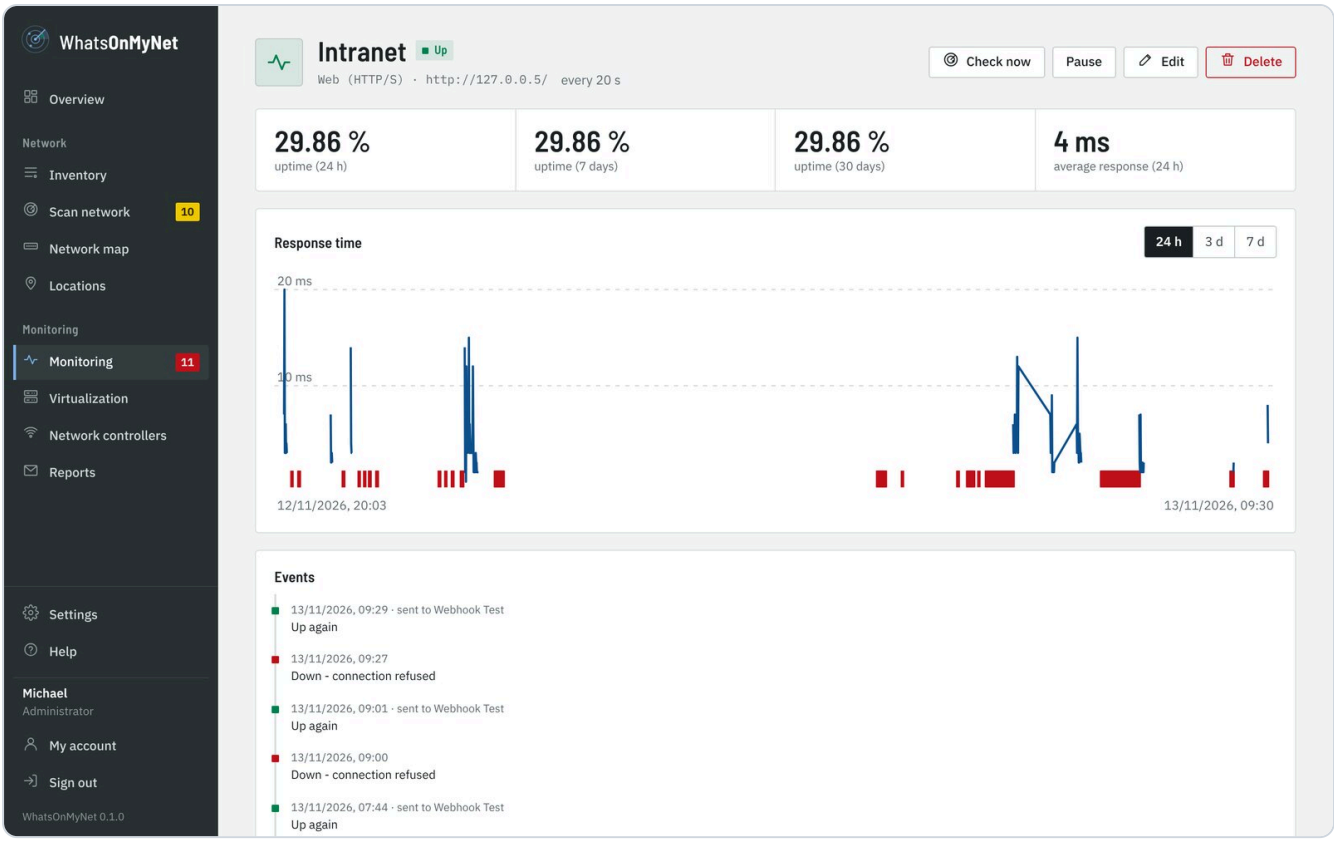


Monitoring

A monitor checks regularly (20 seconds to 24 hours) and reports only after several failures in a row. Kinds:

- **Web (HTTP/S)** with status codes, keyword and certificate expiry (warning e.g. 30, 14 and 7 days before)
- **TCP port** and **ping**
- **Docker container** (through a socket proxy, *Settings → Docker*): running, restarting all the time, health check "unhealthy"
- **Databases**: MariaDB/MySQL, PostgreSQL, Redis/Valkey - sign-in and one harmless query
- **Network drive (SMB)**: Windows shares and NAS - see below
- **Sensor (device value)**, **SNMP value (OID)** and **Virtualization** (Proxmox, VMware, Hyper-V …) - see below

On the device page "Monitor" creates a fitting monitor. If the IP of a device changes, its monitors follow. If you rename the device, its monitors and sensors take the new name too, so an alarm names it as it is called now (a monitor with a name of your own without the device name stays as it is).



Monitor with history

Notifications (*Settings → Notifications → Add channel*): choose the service, follow the short guide in the form, save and check with **Test message**.

- **Common:** e-mail (through the SMTP server), Telegram bot, webhook with your own JSON template (e.g. for n8n or a script)
- **Chat and messengers:** Discord, Slack, Microsoft Teams (Workflows), Google Chat, Mattermost, Rocket.Chat, Matrix/Element, Signal (through signal-cli-rest-api), WhatsApp (CallMeBot, to your own number)
- **Push to the phone:** ntfy, Gotify, Pushover, Pushbullet, Home Assistant (companion app)
- **Everything else:** Apprise - a bridge to more than 100 further services (SMS, Threema ...)

Tokens, keys and webhook addresses are stored encrypted and never shown again. Outages arrive with high priority on push services. **Maintenance windows** (once or weekly) suppress alarms.

Dependencies: no alarm flood

When a switch fails, every device behind it is unreachable too. So that twenty messages do not arrive at once, every monitor can **depend on** another one (form → *Depends on*, e.g. the ping of the switch or the internet router).

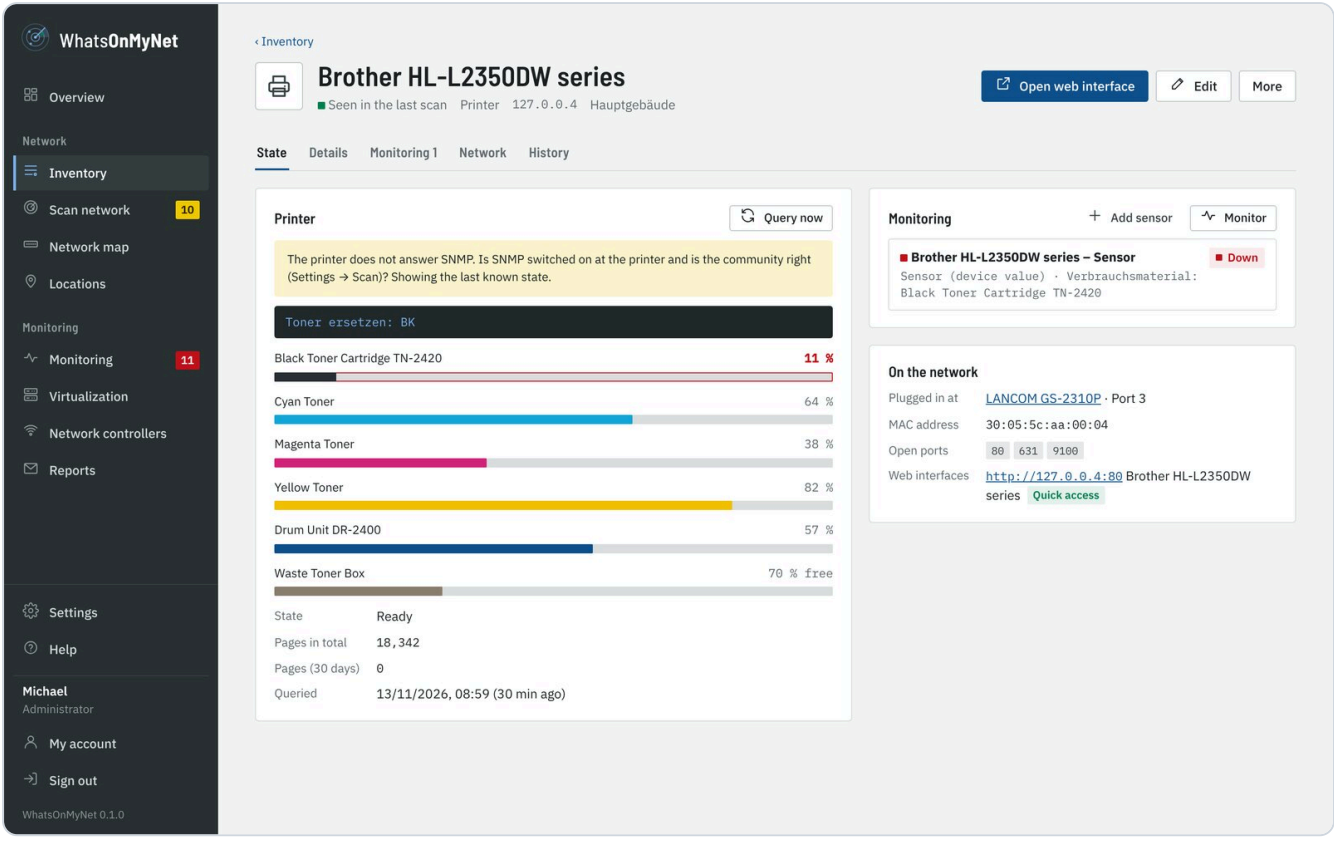
- While the monitor in front is down, the one behind it is a **follow-up outage** (hollow red dot) and sends nothing. Only the cause is reported.
- When both fail at the same time, the one behind waits briefly for the result of the one in front instead of raising its own alarm.

- When the monitor in front is back, the ones behind are checked again right away. Whatever still fails has a problem of its own and is reported as usual.
- Chains are possible (internet router → core switch → floor switch → printer). Circles are refused.

Automatically by switch port: When a switch is queried by SNMP, WhatsOnMyNet knows which switch a device is plugged in at. When you **Monitor** a device, the monitor of its switch is already chosen as dependency. The page of the switch monitor also tells how many monitored devices are plugged in at it and depend on nothing yet. **Put them behind this monitor** sets them all at once.

Printers, switches and NAS

Devices of type *Printer*, *Switch* and *NAS* are queried via SNMP (read-only). Set the community under *Settings* → *Scan*; SNMP v1/v2c has to be on at the device.

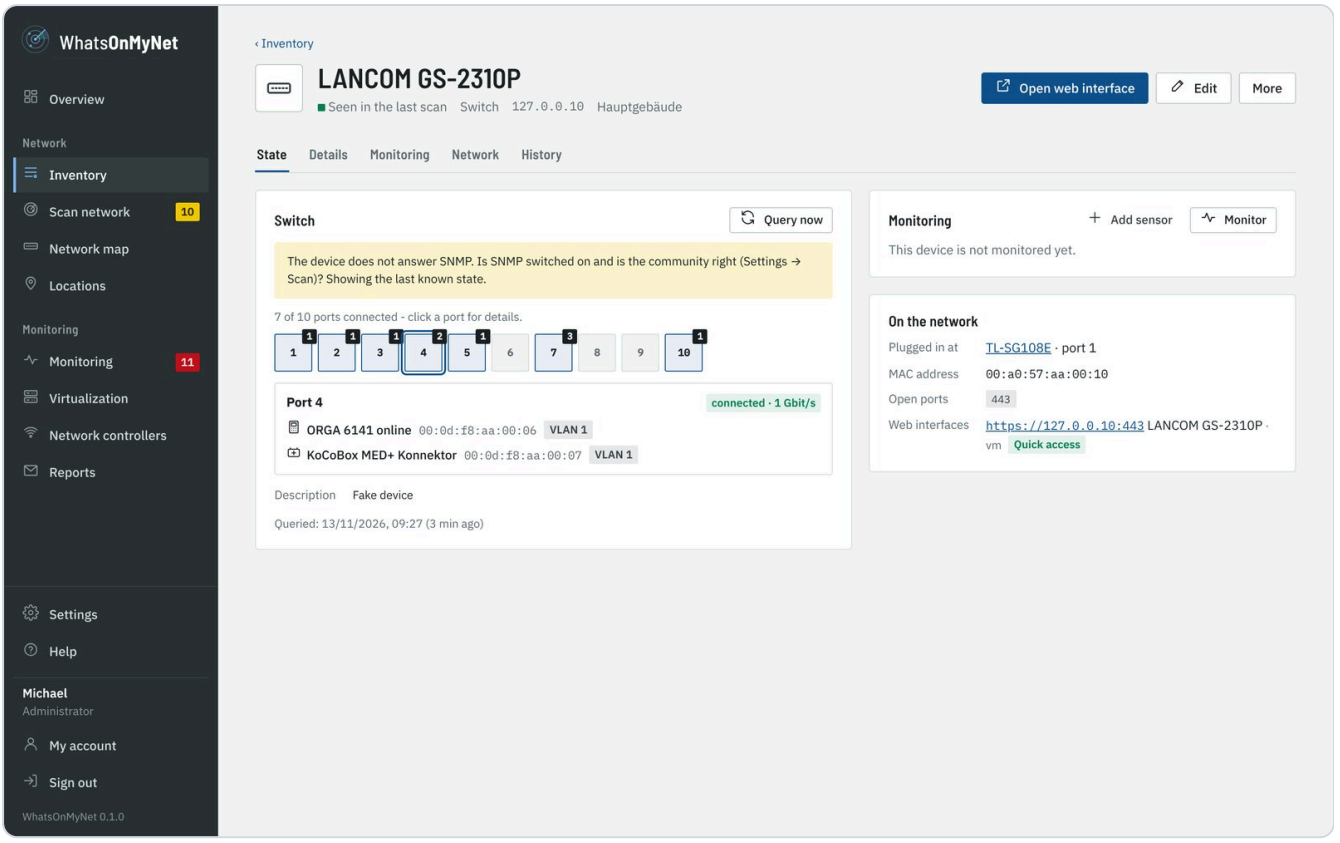


Printer

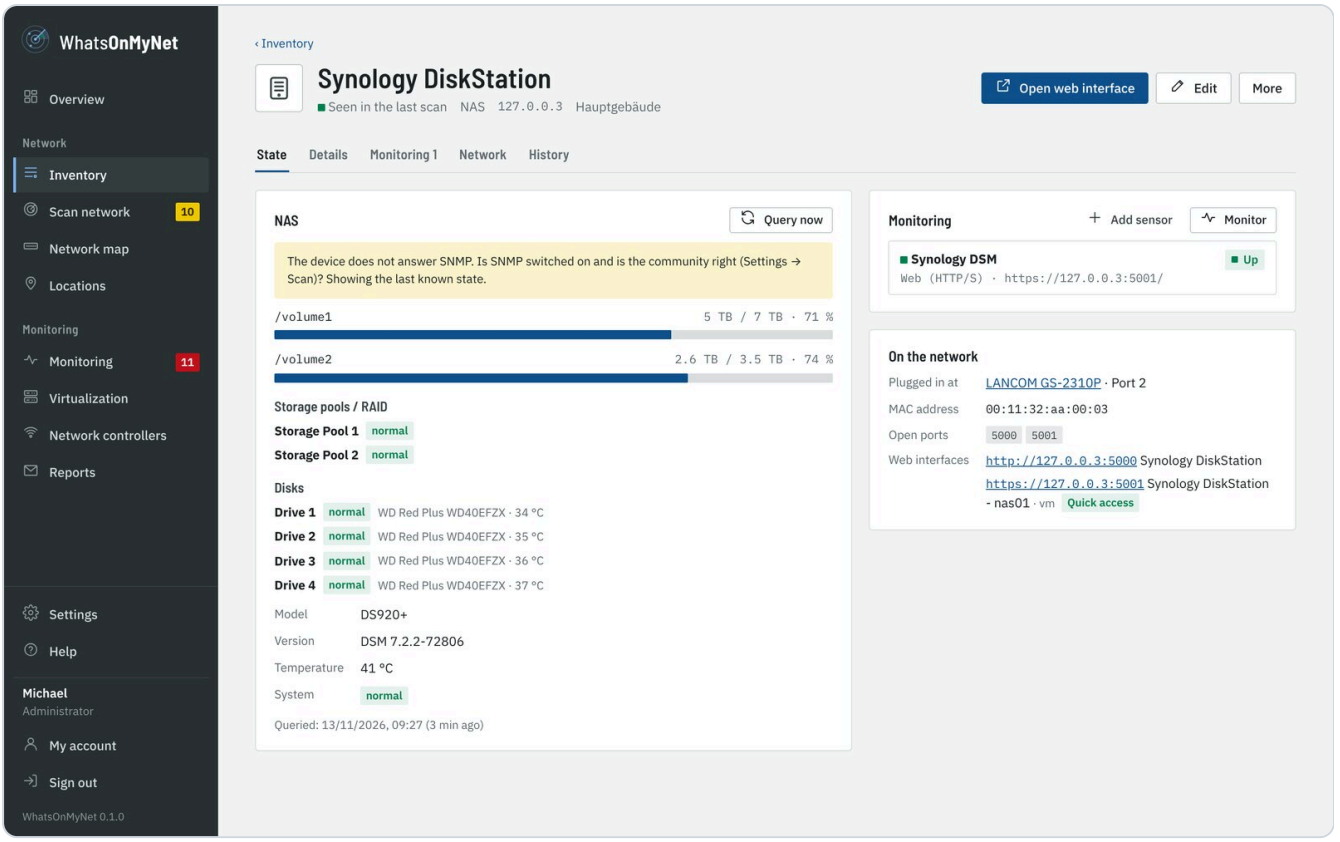
- **Printer**: toner or ink per colour, drum, waste toner, page counter, faults and display text. Below 15 % a notice is sent - once per cartridge.
- **Switch**: ports as tiles, which devices hang at which port (VLAN-aware), LLDP neighbours, firmware. Every device page shows "Plugged in at".
- **NAS**: volumes; on Synology also disks, RAID, temperature, fans, power supply and DSM update. Problems are reported - and again when everything is fine.
- **Taken into the inventory**: what the device says about itself - serial number, model, firmware or operating system (e.g. DSM version), for Synology/QNAP the manufacturer - goes right into

the details of the device and stays current (e.g. after a firmware update). What you entered yourself with another value stays. The history shows every takeover.

- **Device exchanged**: if the device at the same IP reports another serial number, it is new hardware. Serial number, model, manufacturer and firmware then come from the new device, the old MAC address is dropped and replaced by the new one on the next scan (instead of a "new device"). The history notes the exchange - please check purchase date and warranty yourself.



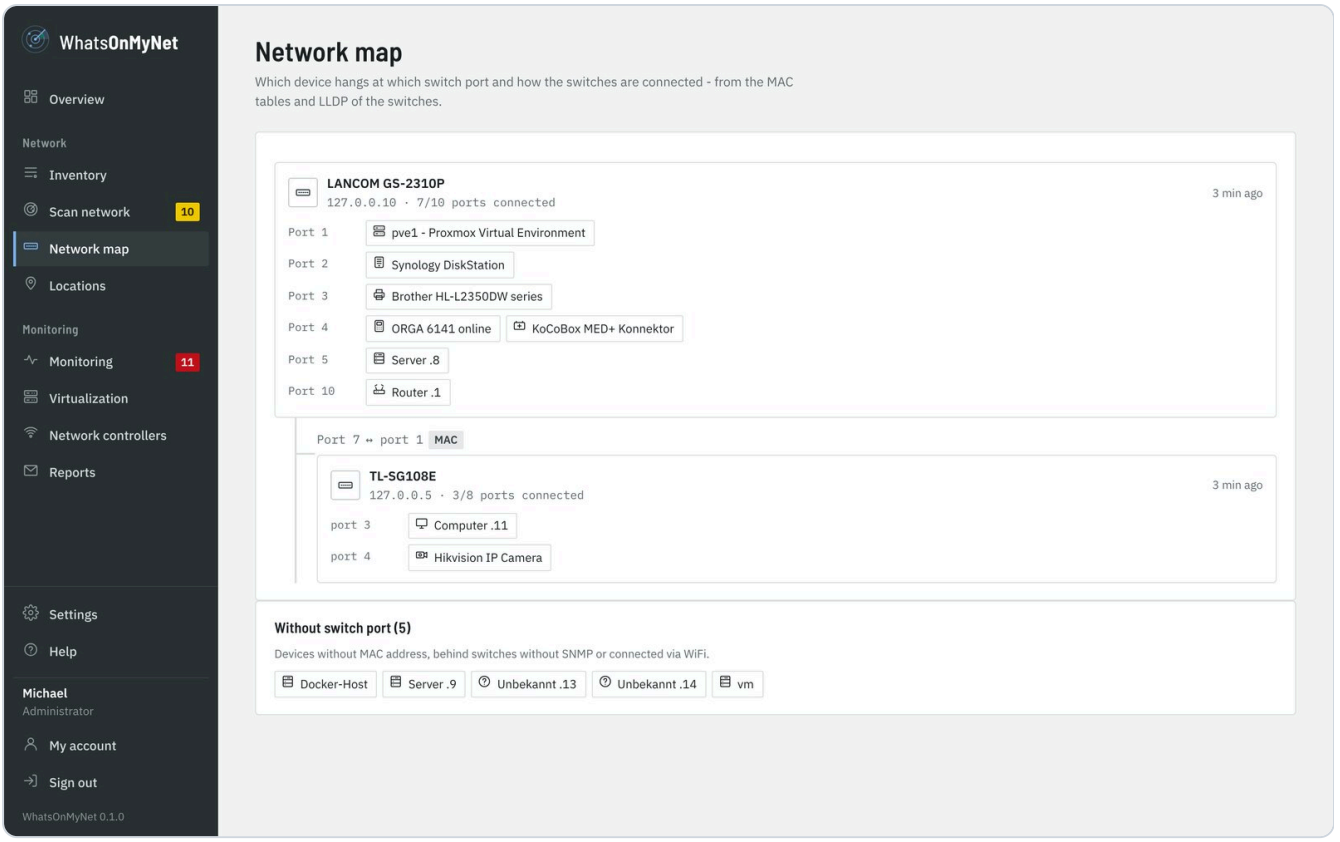
Switch



NAS

Network map

The *Network map* shows from MAC tables and LLDP which device hangs at which port and how the switches are connected. Simple switches without SNMP show up as a branch behind the port they are plugged into. Behind routers the scan reads the ARP table of the routers via SNMP, so the MAC address stays the identity there too.



Network map

Sensors

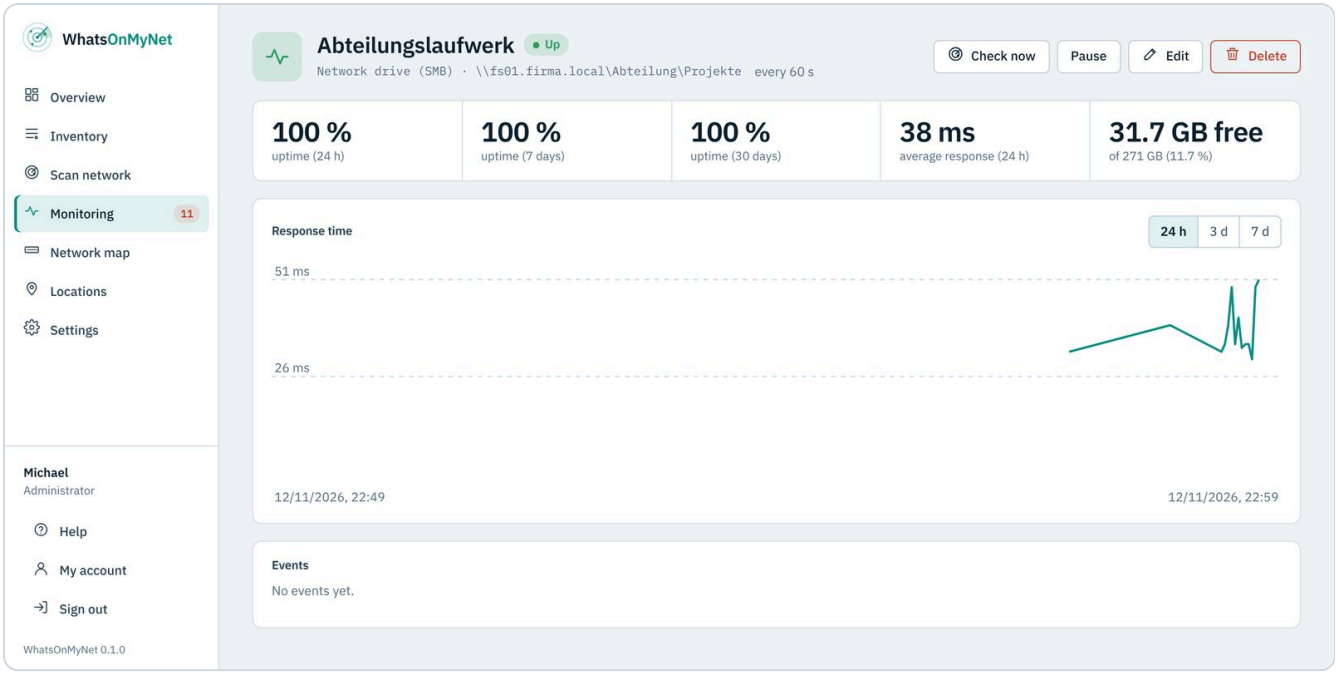
Every value a device delivers can become a monitor of its own - with history, alarm and maintenance windows. On the device page "Add sensor", then choose value and condition, e.g.:

- "toner cyan less than or equal 10 %"
- "usage /volume1 greater than or equal 85 %"
- "temperature drive 2 greater than 50 °C"
- "port 10 (uplink) connected"

The kind **SNMP value (OID)** reads any SNMP value (e.g. runtime left of a UPS) and compares with less, greater, equal or "contains text".

In the monitor list the column **Now** shows the value right away: number and unit, a small bar for percentages, "used of total" for storage and RAM, and the alarm limit ("alarm from 90 %"). Close to the limit the value turns yellow, at the alarm red. Nodes and VMs show their state with CPU and RAM, network drives the free space.

Network drives (SMB)



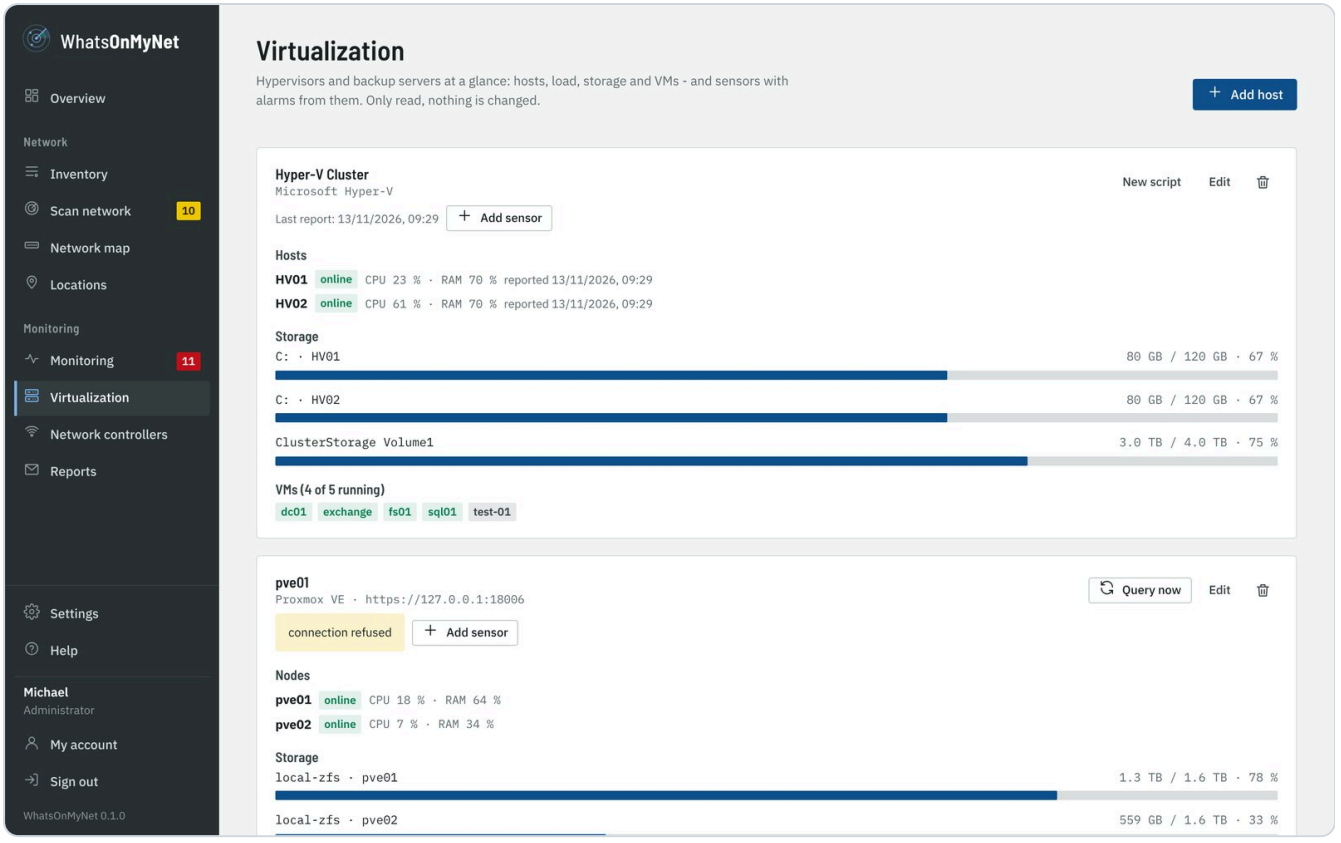
Network drive

The kind **Network drive (SMB)** really signs in to a share - not just to the open port. So you also notice when the password expired, the rights changed or a folder is missing.

1. Create an account of its own with **read rights** on the share (in AD e.g. `COMPANY\monitor`, on a NAS a local user).
2. *Add monitor* → kind **Network drive (SMB)**. Just paste the path, e.g. `\\fs01\Department\Projects` - server, share and folder are taken from it.
3. Enter user, domain (empty for NAS accounts) and password. Optional: warn below … % or … GB free.

The check runs over SMB 2/3: sign in, read the share (or the folder or file), ask for the free space. WhatsOnMyNet writes nothing to the share. The detail page shows the free space; reports have the section *Network drives* for it.

Proxmox VE and Backup Server



Proxmox

WhatsOnMyNet reads Proxmox VE and the Proxmox Backup Server through their built-in interface (API). Nothing is installed on Proxmox. WhatsOnMyNet gets a user of its own there that may **only read**, and a key for it (API token). The dialog *Monitoring → Virtualization → Add host → Proxmox VE* shows the same steps, with a button to copy.

Step 1: prepare Proxmox (once)

With commands (recommended) - Proxmox VE

1. In Proxmox click a node on the left and open **>_ Shell** at the top. One node is enough - in a cluster it applies to all.
2. Copy these three lines, paste them into the shell and press Enter:

```
pveum user add whatsonmynet@pve --comment "WhatsOnMyNet (read only)"
pveum acl modify / --users whatsonmynet@pve --roles PVEAuditor
pveum user token add whatsonmynet@pve womn --privsep 0
```

3. The output shows a table. The row `value` holds the **secret** (a long string with dashes). Copy and keep it - Proxmox shows it only this once.

With commands - Proxmox Backup Server

1. In the Backup Server open *Administration* → *Shell* on the left.
2. Paste these four lines and press Enter:

```
proxmox-backup-manager user create whatsonmynet@pbs --comment "WhatsOnMyNet (read only)"
proxmox-backup-manager acl update / Audit --auth-id whatsonmynet@pbs
proxmox-backup-manager user generate-token whatsonmynet@pbs womn
proxmox-backup-manager acl update / Audit --auth-id 'whatsonmynet@pbs!womn'
```

3. After the third command `"value"` holds the **secret**. Copy it - it is shown only once.

By clicking - Proxmox VE

1. *Datacenter* → *Permissions* → *Users* → *Add*: user name `whatsonmynet`, realm *Proxmox VE authentication server*, any password → *Add*.
2. *Datacenter* → *Permissions* → *Add* → *User Permission*: path `/`, user `whatsonmynet@pve`, role `PVEAuditor` → *Add*.
3. *Datacenter* → *Permissions* → *API Tokens* → *Add*: user `whatsonmynet@pve`, token ID **only** `womn` (no @ or !), **untick** *Privilege Separation* → *Add*.
4. Proxmox now shows the *Secret* - only this once. Copy and keep it.

On the Backup Server it works the same under *Configuration* → *Access Control*: user `whatsonmynet` (realm *Proxmox Backup authentication server*), tab *API Token* with token name `womn`, then under *Permissions* the role `Audit` on the path `/` twice - once for the user `whatsonmynet@pbs`, once for the token `whatsonmynet@pbs!womn`.

Step 2: enter it in WhatsOnMyNet

1. *Monitoring* → *Virtualization* → **Add host** → *Proxmox VE* or *Proxmox Backup Server*.
2. Give it a name.
3. Enter the **address** the way you open Proxmox in the browser, e.g. `https://192.168.1.10:8006` (Backup Server: port `8007`).
4. Click **Fetch certificate**. WhatsOnMyNet shows the fingerprint of the Proxmox certificate. Compare it - Proxmox VE: click the node → *System* → *Certificates* → row `pve-ssl.pem`; Backup Server: *Dashboard* → *Show Fingerprint*. If it matches, **Fingerprint matches - trust**.
5. The **token ID** is already filled in (`whatsonmynet@pve!womn` or `whatsonmynet@pbs!womn`). Paste the copied secret into **Token secret**.
6. **Save and connect.** After a few seconds nodes, storage, VMs and containers and the backups appear.

Adding sensors

In the detail view **Add sensor** - e.g. "VM 101 running", "storage local-lvm below 85 %" or "last backup older than 26 hours". Every sensor is a monitor with history and notification.

If something does not work

- **"tokenid: value does not match the regex pattern" in Proxmox**: when adding the token enter only `womn` - without user, @ and !.
- **"sign-in refused (401)"**: token ID or secret are wrong. Delete the token in Proxmox, add it again and enter the new secret.
- **Notice "The token lacks read rights"**: usually *Privilege Separation* stayed ticked on the token. The notice shows two commands - paste them into the Proxmox shell, Enter, then **Query now**.
- **"certificate changed"**: Proxmox got a new certificate. *Edit* → **Fetch certificate** → confirm.

More hypervisors: VMware, XCP-ng, Hyper-V, Nutanix, oVirt, KVM

Under *Monitoring* → *Virtualization* → *Add host* further hypervisors are offered beside Proxmox. All appear the same way: hosts with CPU and RAM, storage, VMs - and sensors can be added from everything ("host online", "CPU esx01", "storage vsan", "VM dc01 running"). It only reads.

Asked through their interface - an account that may only read is enough; enter the address, **Fetch certificate**, compare the fingerprint, enter the account:

Hypervisor	Account with read rights	Address
VMware ESXi / vCenter	user with role *Read-only*, given at the top of the inventory, "propagate to children"	vCenter or ESXi, e.g. `https://vcenter.firma.local`
XCP-ng / XenServer	with AD: role *read-only*; without AD there is only `root`	pool master, e.g. `https://xcp01.firma.local`
Nutanix AHV	Prism user with role *Viewer*	Prism with port, e.g. `https://prism.firma.local:9440`
oVirt / RHV / OLVM	`whatsonmynet@internal` with role *ReadOnlyAdmin* (commands are in the dialog)	engine, e.g. `https://engine.firma.local`

Hyper-V and KVM/libvirt report on their own. Neither has an interface to ask. Instead a small script on the host sends load, volumes or storage pools and VMs every 5 minutes. It only reads and needs no open port on the host.

1. *Add host* → **Microsoft Hyper-V** or **KVM / libvirt**, give a name, save.
2. The dialog shows the script with a key of its own (only this once) - **Download script**.
3. Run it once, it sets itself up. **Hyper-V** as administrator: `powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\womn-hyperv.ps1` (scheduled task *WhatsOnMyNet-HyperV*, every 5 minutes as SYSTEM). In a cluster on every node - all nodes land in the same entry. **KVM:** `sh womn-libvirt.sh` (cron entry, every 5 minutes). It needs `virsh`, `awk` and `curl`; it reads with `virsh -r`. Removing again: the **uninstall script** from the dialog or *Settings* → *Machine scripts* - it also removes the setup of version 0.6.0.
4. After a few seconds the host appears. If it does not report for more than 15 minutes it counts as offline - a sensor "host online" then raises the alarm.

If the key is lost or should be renewed: **New script** - the old one stops working then.

With the self-signed certificate of WhatsOnMyNet the script trusts exactly this certificate (built in); with a certificate of a real CA it checks as usual.

Network controllers: UniFi, Omada, MikroTik

Under *Monitoring → Network controllers → Add controller* WhatsOnMyNet reads the widespread network systems - with an account that may **only read**; enter the address, **Fetch certificate**, compare the fingerprint, enter the account. The dialog shows the steps for each system.

| System | Account | Address | |---|---|---| | **UniFi Network** - UniFi OS consoles (UDM, UCG, Cloud Key Gen2+) or the installed Network app | local account (without two-factor) with role *View only*/*Site Viewer* | console, or `https://server:8443` | | **TP-Link Omada** - software controller, OC200/OC300 | account with role *Viewer* | usually `https://server:8043` | | **MikroTik RouterOS 7** - every router/switch on its own | user in a group with `read,api,rest-api` | router with service *www-ssl* |

The page shows all access points, switches and gateways of each controller with model, IP, firmware (with "Update" when a new one is ready), clients and state.

Matched with the inventory: every device that is in the inventory by its MAC address gets model, serial number, firmware, manufacturer and the right **type** from the controller - by the same rules as for SNMP (empty or still device-reported details are filled or kept current, your own different entries stay, an exchange is recognised). Devices that are not in the inventory yet carry the note *not in the inventory* - adopt them by scan.

Sensors: "AP hall online", "Clients on AP hall", "CPU ...", "RAM ...", "Clients in total" and "Devices with firmware update".

The **scan** now also recognises these devices better by their model names: UniFi (USW = switch, UAP/U6/U7 = access point, UDM/UCG/USG = gateway, Network app = server), Omada (EAP = access point, ER = gateway, OC200/OC300 = controller), MikroTik (CRS = switch, cAP/wAP = access point) and Aruba Instant On.

Windows and Linux machines by script (without agent)

Servers and PCs can be monitored without an agent: a small script on the machine reports **CPU, RAM, drives, uptime and services** every 5 minutes. It installs nothing, loads nothing, opens no port and needs no account - it only talks to WhatsOnMyNet. On Windows it is a PowerShell script (runs as SYSTEM), on Linux a POSIX sh script (needs `sh`, `awk` and `curl`, root is not needed).

A single machine: *Monitoring* → **Add monitor** → **Machine by script (Windows/Linux)** → choose the machine. On the device page the same works with **Monitor** (kind of check *Machine by script*) or *More* → **Monitor with script**. Then Windows or Linux → **Create script**.

1. **Download script** (it holds the address and the key - shown only this once).

2. Run it once - it sets itself up: - **Windows** as administrator: `powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\womn-host.ps1` - it puts itself into `C:\ProgramData\WhatsOnMyNet` (readable only for SYSTEM and administrators) and adds the scheduled task *WhatsOnMyNet-Host*. - **Linux**: `sh womn-host.sh` - as root into `/usr/local/sbin` with `/etc/cron.d/whatsonmynet-host`, otherwise into `~/.local/bin` with an entry in the own crontab.
3. After a few seconds the device page shows the section **System**. **Add sensor** turns it into monitors with alarms: "Machine reports", "CPU load", "Memory", "Drive C:", "Failed services" or "Service ... running".

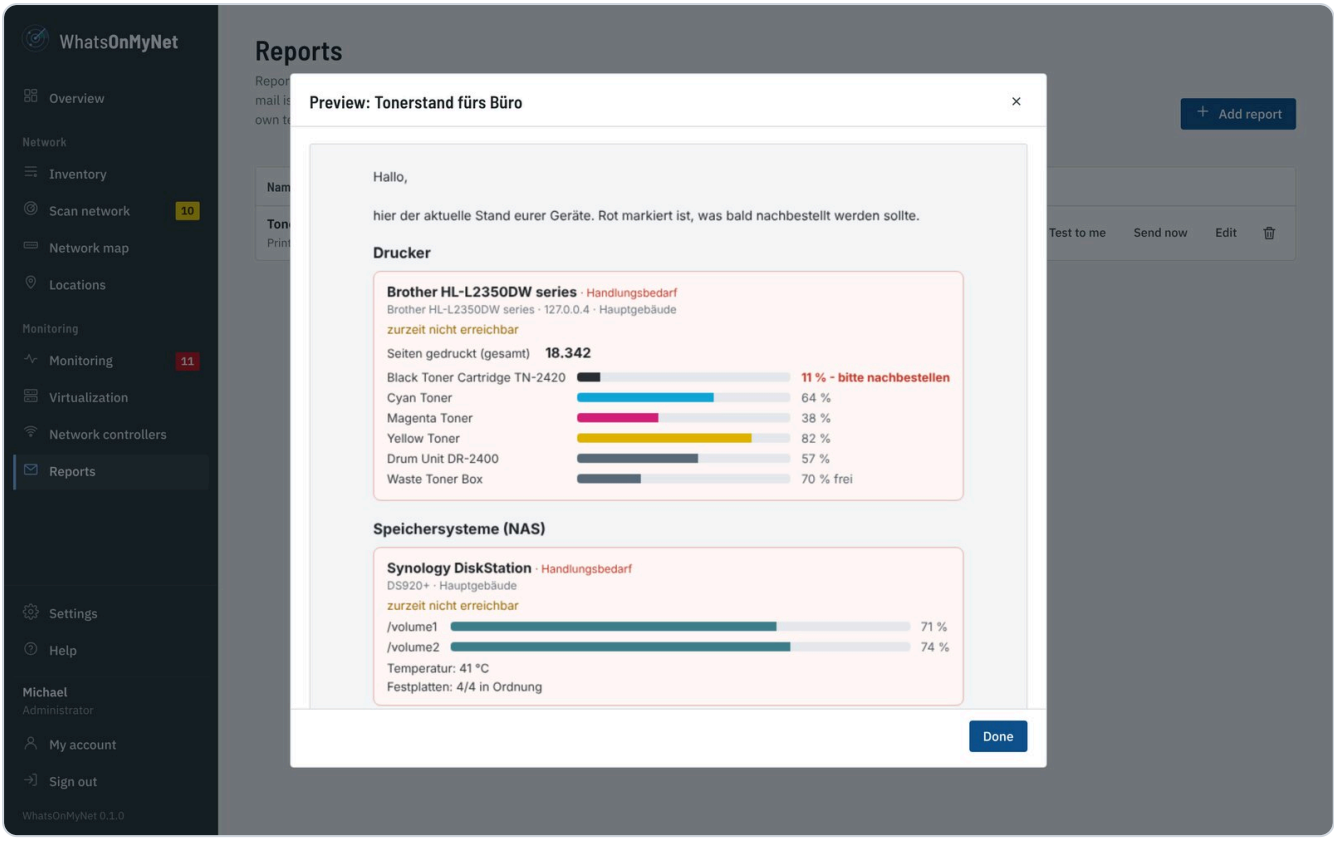
Many machines at once: *Settings* → *Machine scripts* → **Shared script**. You roll out this one script by software distribution, a startup script by group policy or Ansible. Each report is matched to its device in the inventory (MAC address, machine name, IP). Machines that are not in the inventory yet show up there under *Unknown machines* - nothing is created on its own.

Failed services: Windows - services that start automatically and stopped with an error code (trigger-start services that stopped normally do not count); Linux - systemd services in the state *failed*.

Removing again: every script comes with an **uninstall script** (in the script dialog under *Remove again* and under *Settings* → *Machine scripts*). It removes the scheduled task or cron entry and the script - run or distribute it the same way. Afterwards the machine is as before. Deleting or recreating a script under *Settings* → *Machine scripts* locks the old key.

If a machine does not report for 15 minutes it counts as offline - the sensor "Machine reports" then raises the alarm.

Reports



Report

Under *Monitoring → Reports* reports go out by e-mail at set times - e.g. the toner levels to the office every Monday at 8:00.

- Content: printers, NAS, switches, network drives; can be limited to locations; optionally only devices that need action.
- **Devices**: all devices of the chosen content - or only chosen ones, e.g. a report of its own for the printer in the front office.
- Printers show the **pages printed** (in total and in the report period, e.g. "in the last 7 days") and - if the printer reports it - how many pages the toner still lasts.
- The mail is **neutral**: no link, no program name - only your sender name, subject (`{week}` , `{date}`), introduction, the devices and your closing.
- "Preview", "Test to me" and "Send now" right in the list.

Users and rights

- **Roles**: *Administrator* (everything), *Editor* (scan, adopt, maintain, monitors), *Read only*.
- **Locations**: editors and viewers can be limited to locations - they then only see and change devices, scans and monitors of these.
- **Active Directory**: sign-in with the Windows account, see the next section.

- **Two-factor sign-in** (*My account*): scan the QR code with an authenticator app, confirm a code, keep the recovery codes. It can be made mandatory for administrators under *Settings* → *General*.

Sign-in with Active Directory

Staff sign in with their Windows account. AD groups decide which role they get. Local accounts keep working; for the same name the local account wins, so the local admin stays the emergency access.

Step 1: prepare AD (once)

1. Create a **service account**, e.g. `svc-womn`. It needs no special rights, a normal domain account with a password is enough.
2. Create up to three **groups**, e.g. `WOMN-Admins`, `WOMN-Editors` and `WOMN-Read`, and put the staff into them. Groups in groups count.

Step 2: enter it in WhatsOnMyNet

1. *Settings* → *Active Directory*.
2. **Server**: name of the domain controller as it is in the certificate, e.g. `dc01.company.local`. Encryption **LDAPS** (port 636) or **StartTLS** (port 389). Without encryption it does not work.
3. **Certificate**: leave trust on *Fetched certificate* and click **Fetch certificate from DC**. Compare the fingerprint with the certificate on the DC (`certlm.msc` → *Personal* → open the certificate → *Details* → *Thumbprint*) and confirm. If the root CA is missing, WhatsOnMyNet gets it itself (via the AIA address in the certificate or, after confirming, from AD).
4. **Directory**: leave the base DN empty, the test finds it. Enter the service account, e.g. `svc-womn@company.local`, and its password.
5. **Groups**: simply enter the group names, e.g. `WOMN-Admins`. Members of several groups get the highest role; members of none cannot sign in.
6. **Test connection**, best with your own user name. Every step gets ✓ or × with the reason: connection, service account, base DN, groups, user, role and, if you like, the password.
7. When everything is ✓: tick *Allow sign-in through Active Directory* and **Save**.

If something does not work

- **"The certificate of the DC is not trusted"**: *Fetch certificate from DC* and confirm.
- **"The name in the certificate does not fit"**: enter the name from the certificate as domain controller (fetching shows it). Only if you reach the DC by its IP address: switch *Check the server name in the certificate* off.
- **"Service account: sign-in refused"**: name or password of the service account wrong, or the account is locked.
- **"Group not found"**: check the spelling of the group in AD.
- **"No role"**: the user is in none of the groups entered.

Scan agent for branch offices

For a branch office a second container runs there as scan agent. Under *Settings* → *Scan agents* add an agent for the location; put the `.env` shown next to `docker-compose.agent.yml` from the release and start `docker compose -f docker-compose.agent.yml up -d`. The agent opens no port, fetches jobs over HTTPS and reports only addresses of its location.

Backup and updates

- Every night WhatsOnMyNet backs up the database to `data/backups/`; additionally before every database update.
- *Settings* → *Backup*: download a backup, back up now, restore a backup (the current state is backed up first, then the program restarts).
- **Keep `data/secret.key` (or `WOMN_SECRET`) as well** - without it stored passwords in a backup are unreadable.
- Updates: in Dockhand/Watchtower or `docker compose pull && docker compose up -d`. Once a month a security update with new packages is released.

Troubleshooting

- **Interface not reachable**: open the host firewall for the HTTPS port. With host networking Docker/Dockhand shows no port mapping - that is correct.
- **No printer/switch data**: switch on SNMP at the device, check the community under *Settings* → *Scan*, "Query now" on the device page.
- **Locked out**: `docker compose exec whatsonmynet womn-admin reset <user>` sets a new password, makes the account administrator again and switches two-factor sign-in off. `womn-admin create <user>` adds a new administrator.
- **Network drive reports "protocol not supported"**: the device only speaks the old SMB1 - it is not used for security reasons. Switch on SMB2/3 at the device (Synology: *File Services* → *SMB* → *Advanced*).
- **Log**: *Settings* → *Log* shows who changed what; technical messages are in the container log.